DS homework programs exercise 5 (2025115100)

1. Polynomial Representation Using Linked List — Addition and Subtraction

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
   int coeff;
   int exp;
   struct Node *next;
};
struct Node *insert(struct Node *head, int c, int e) {
   struct Node *n = malloc(sizeof(struct Node));
   n->coeff = c;
   n->exp = e;
   n->next = NULL;
   if (head == NULL)
      return n;
   struct Node *temp = head;
   while (temp->next != NULL)
      temp = temp->next;

   temp->next = n;
   return head;
}
struct Node *add(struct Node *p, struct Node *q, int sign) {
   struct Node *r = NULL;

   while (p != NULL && q != NULL) {
      if (p->exp == q->exp) {
         int c = p->coeff + sign * q->coeff;
         if (c != 0)
            r = insert(r, c, p->exp);
```

```
            p = p->next;
            q = q->next;
        } else if (p->exp > q->exp) {
            r = insert(r, p->coeff, p->exp);
            p = p->next;
        } else {
            r = insert(r, sign * q->coeff, q->exp);
            q = q->next;
        }
    }
    while (p != NULL) {
        r = insert(r, p->coeff, p->exp);
        p = p->next;
    }
    while (q != NULL) {
        r = insert(r, sign * q->coeff, q->exp);
        q = q->next;
    }
    return r;
}
void display(struct Node *p) {
    while (p != NULL) {
        printf("%dx^%d", p->coeff, p->exp);
        p = p->next;
        if (p != NULL)
            printf(" + ");
    }
    printf("\n");
}
int main() {
    struct Node *p = NULL, *q = NULL, *sum, *diff;
```

```
    p = insert(p, 5, 2);
    p = insert(p, 4, 1);
    p = insert(p, 2, 0);
    q = insert(q, 3, 2);
    q = insert(q, 2, 1);
    q = insert(q, 1, 0);
    printf("First Polynomial: ");
    display(p);
    printf("Second Polynomial: ");
    display(q);
    sum = add(p, q, 1);
    diff = add(p, q, -1);
    printf("Addition: ");
    display(sum);

    printf("Subtraction: ");
    display(diff);
    return 0;
}
```

Output

```
First Polynomial: 5x^2 + 4x^1 + 2x^0
Second Polynomial: 3x^2 + 2x^1 + 1x^0
Addition: 8x^2 + 6x^1 + 3x^0
Subtraction: 2x^2 + 2x^1 + 1x^0
```

2. Find Previous Node of Circular Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
};

int main() {
```

```
    struct Node *head, *second, *third, *fourth, *temp;
    int key;

    head = malloc(sizeof(struct Node));
    second = malloc(sizeof(struct Node));
    third = malloc(sizeof(struct Node));
    fourth = malloc(sizeof(struct Node));

    head->data = 10;
    head->next = second;

    second->data = 20;
    second->next = third;

    third->data = 30;
    third->next = fourth;

    fourth->data = 40;
    fourth->next = head;

    printf("Circular Linked List: 10 20 30 40\n");
    printf("Enter element to find previous node: ");
    scanf("%d", &key);

    temp = head;

    do {
      if (temp->next->data == key) {
        printf("Previous node: %d\n", temp->data);
        return 0;
      }
      temp = temp->next;
    } while (temp != head);

    if (head->data == key) {
      printf("Previous node: %d\n", fourth->data);
    } else {
      printf("Element not found\n");
    }

    return 0;
}
```

Output

```
Circular Linked List: 10 20 30 40
Enter element to find previous node: 30
Previous node: 20
```

3. Insert at a Given Position in a Circular Linked List

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
};

int main() {
   struct Node *head, *second, *third, *newnode, *temp;
   int pos, value, i;

   head = malloc(sizeof(struct Node));
   second = malloc(sizeof(struct Node));
   third = malloc(sizeof(struct Node));

   head->data = 10;
   head->next = second;

   second->data = 20;
   second->next = third;

   third->data = 30;
   third->next = head;

   printf("Original List: 10 20 30\n");
   printf("Enter position: ");
   scanf("%d", &pos);

   printf("Enter value: ");
   scanf("%d", &value);

   if (pos < 1) {
      printf("Invalid position\n");
      return 0;
   }

   newnode = malloc(sizeof(struct Node));
   newnode->data = value;

   if (pos == 1) {
      temp = head;
      while (temp->next != head)
         temp = temp->next;

      newnode->next = head;
      temp->next = newnode;
      head = newnode;
```

```
        } else {
            temp = head;

            for (i = 1; i < pos - 1 && temp->next != head; i++)
                temp = temp->next;

            if (i != pos - 1) {
                printf("Invalid position\n");
                return 0;
            }

            newnode->next = temp->next;
            temp->next = newnode;
        }

        printf("Updated List: ");
        temp = head;

        do {
            printf("%d ", temp->data);
            temp = temp->next;
        } while (temp != head);

        printf("\n");
        return 0;
    }
```

Output

Original List: 10 20 30

Enter position: 2

Enter value: 15

Updated List: 10 15 20 30

4. Find the Largest and Second Largest Elements in a Queue

```
#include <stdio.h>

int main() {

    int queue[100], n, i;

    int largest, second;

    printf("Enter number of elements: ");

    scanf("%d", &n);

    if (n < 2 || n > 100) {

        printf("Enter at least 2 and at most 100 elements\n");

        return 0;

    }
```

```
    printf("Enter queue elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &queue[i]);
    largest = second = -2147483647 - 1;
    for (i = 0; i < n; i++) {
        if (queue[i] > largest) {
            second = largest;
            largest = queue[i];
        } else if (queue[i] > second && queue[i] < largest) {
            second = queue[i];
        }
    }
    if (second == -2147483647 - 1)
        printf("Second largest distinct element does not exist\n");
    else
        printf("Second largest element: %d\n", second);
    return 0;
}
```

output

Enter number of elements: 5

Enter queue elements: 10 40 20 50 30

Second largest element: 40

5. Insert the Third Element and Check Whether the First and Third Elements Are the Same in a Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *head, *second, *third, *temp;
    int value, count = 1;
```

```
    head = malloc(sizeof(struct Node));
    second = malloc(sizeof(struct Node));

    head->data = 10;
    head->next = second;

    second->data = 20;
    second->next = NULL;

    printf("Enter third element: ");
    scanf("%d", &value);

    third = malloc(sizeof(struct Node));
    third->data = value;
    third->next = NULL;

    second->next = third;

    printf("Linked List: ");

    temp = head;
    while (temp != NULL) {
       printf("%d ", temp->data);
       temp = temp->next;
    }

    printf("\n");

    if (head->data == third->data)
       printf("First and third elements are same\n");
    else
       printf("First and third elements are not same\n");
    return 0;
}
```

Output 1

```
Enter third element: 10
Linked List: 10 20 10
First and third elements are same
```

Output 2

```
Enter third element: 30
Linked List: 10 20 30
First and third elements are not same
```

6. Implement two stacks in one array and traverse the array

```
#include <stdio.h>
#define SIZE 10

int arr[SIZE];
```

```
int top1 = -1;
int top2 = SIZE;

void push1(int value)
{
   if (top1 + 1 == top2)
      printf("Stack Overflow\n");
   else
      arr[++top1] = value;
}

void push2(int value)
{
   if (top1 + 1 == top2)
      printf("Stack Overflow\n");
   else
      arr[--top2] = value;
}

void display()
{
   int i;

   printf("Stack 1: ");
   for (i = top1; i >= 0; i--)
      printf("%d ", arr[i]);

   printf("\nStack 2: ");
   for (i = top2; i < SIZE; i++)
      printf("%d ", arr[i]);

   printf("\n");
}

int main()
{
   push1(10);
   push1(20);
   push1(30);

   push2(40);
   push2(50);
   push2(60);

   display();

   return 0;
}
```

Output
Stack 1: 30 20 10
Stack 2: 60 50 40

7. Flatten a linked list while maintaining sorted order

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
   int data;
   struct Node *next;
   struct Node *bottom;
};

struct Node *newNode(int data)
{
   struct Node *node = malloc(sizeof(struct Node));
   node->data = data;
   node->next = NULL;
   node->bottom = NULL;
   return node;
}

struct Node *merge(struct Node *a, struct Node *b)
{
   if (a == NULL)
      return b;

   if (b == NULL)
      return a;

   struct Node *result;

   if (a->data <= b->data)
   {
      result = a;
      result->bottom = merge(a->bottom, b);
   }
   else
   {
      result = b;
      result->bottom = merge(a, b->bottom);
   }

   result->next = NULL;
   return result;
}
```

```
struct Node *flatten(struct Node *root)
{
   if (root == NULL || root->next == NULL)
      return root;

   root->next = flatten(root->next);
   root = merge(root, root->next);

   return root;
}

void display(struct Node *root)
{
   while (root != NULL)
   {
      printf("%d", root->data);
      if (root->bottom != NULL)
         printf(" -> ");
      root = root->bottom;
   }
   printf("\n");
}

int main()
{
   struct Node *head = newNode(5);
   head->bottom = newNode(7);
   head->bottom->bottom = newNode(8);

   head->next = newNode(10);
   head->next->bottom = newNode(20);

   head->next->next = newNode(19);
   head->next->next->bottom = newNode(22);

   head->next->next->next = newNode(28);
   head->next->next->next->bottom = newNode(40);
   head->next->next->next->bottom->bottom = newNode(45);

   head = flatten(head);

   display(head);

   return 0;
}
```

Output

5 -> 7 -> 8 -> 10 -> 19 -> 20 -> 22 -> 28 -> 40 -> 45

8. Merge k sorted linked lists into a single sorted list

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
   int data;
   struct Node *next;
};

struct Node *newNode(int data)
{
   struct Node *node = malloc(sizeof(struct Node));
   node->data = data;
   node->next = NULL;
   return node;
}

struct Node *merge(struct Node *a, struct Node *b)
{
   struct Node dummy;
   struct Node *tail = &dummy;

   dummy.next = NULL;

   while (a != NULL && b != NULL)
   {
      if (a->data <= b->data)
      {
         tail->next = a;
         a = a->next;
      }
      else
      {
         tail->next = b;
         b = b->next;
      }

      tail = tail->next;
   }

   tail->next = (a != NULL) ? a : b;

   return dummy.next;
}

void display(struct Node *head)
{
   while (head != NULL)
```

```
    {
        printf("%d", head->data);
        if (head->next != NULL)
            printf(" -> ");
        head = head->next;
    }
    printf("\n");
}

int main()
{
    struct Node *a = newNode(1);
    a->next = newNode(4);
    a->next->next = newNode(7);

    struct Node *b = newNode(2);
    b->next = newNode(5);
    b->next->next = newNode(8);

    struct Node *c = newNode(3);
    c->next = newNode(6);
    c->next->next = newNode(9);

    struct Node *result = merge(a, b);
    result = merge(result, c);

    display(result);

    return 0;
}
```

Output

```
1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9
```

9. Swap the kth node from the beginning with the kth node from the end

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

struct Node *newNode(int data)
{
    struct Node *node = malloc(sizeof(struct Node));
    node->data = data;
    node->next = NULL;
    return node;
```

```
}

struct Node *swapNodes(struct Node *head, int k)
{
   int n = 0, i;
   struct Node *temp = head;

   while (temp != NULL)
   {
      n++;
      temp = temp->next;
   }

   if (k > n || k <= 0)
      return head;

   if (2 * k - 1 == n)
      return head;

   struct Node *prev1 = NULL, *curr1 = head;
   struct Node *prev2 = NULL, *curr2 = head;

   for (i = 1; i < k; i++)
   {
      prev1 = curr1;
      curr1 = curr1->next;
   }

   for (i = 1; i < n - k + 1; i++)
   {
      prev2 = curr2;
      curr2 = curr2->next;
   }

   if (prev1 != NULL)
      prev1->next = curr2;
   else
      head = curr2;

   if (prev2 != NULL)
      prev2->next = curr1;
   else
      head = curr1;

   temp = curr1->next;
   curr1->next = curr2->next;
   curr2->next = temp;
```

```
    return head;
}

void display(struct Node *head)
{
    while (head != NULL)
    {
        printf("%d", head->data);
        if (head->next != NULL)
            printf(" -> ");
        head = head->next;
    }
    printf("\n");
}

int main()
{
    struct Node *head = newNode(1);
    head->next = newNode(2);
    head->next->next = newNode(3);
    head->next->next->next = newNode(4);
    head->next->next->next->next = newNode(5);

    printf("Original List: ");
    display(head);

    head = swapNodes(head, 2);

    printf("After Swapping: ");
    display(head);

    return 0;
}
```

Output

Original List: 1 -> 2 -> 3 -> 4 -> 5

After Swapping: 1 -> 4 -> 3 -> 2 -> 5

10. Multiply two numbers represented by two linked lists

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

struct Node *newNode(int data)
{
```

```
    struct Node *node = malloc(sizeof(struct Node));
    node->data = data;
    node->next = NULL;
    return node;
}

unsigned long long multiply(struct Node *a, struct Node *b)
{
    unsigned long long num1 = 0, num2 = 0;
    const unsigned long long MOD = 1000000007ULL;

    while (a != NULL)
    {
        num1 = (num1 * 10 + a->data) % MOD;
        a = a->next;
    }

    while (b != NULL)
    {
        num2 = (num2 * 10 + b->data) % MOD;
        b = b->next;
    }

    return (num1 * num2) % MOD;
}

int main()
{
    struct Node *a = newNode(1);
    a->next = newNode(0);
    a->next->next = newNode(0);

    struct Node *b = newNode(1);
    b->next = newNode(0);

    printf("Product: %llu\n", multiply(a, b));

    return 0;
}
```

Output

Product: 10000